JENGASEC 2026 · APPLICATION BLUE TRACK

Briefing Summary & Team Game Plan

What the competition briefing said, the deadlines that matter, how to prepare to win, and how our four-person team splits the work.

Deadlines at a glance

Deadline	What	Status
Mon 12 Oct, 23:59	Every member: profile, ID upload, Code of Conduct accepted. Captain clears team checklist.	Hard deadline
Tue 13 Oct, 2:00 PM EAT	Build ends. Last **Healthy** image is frozen. Documents due.	Hard deadline, no grace
Before 14 Oct	Download personal WireGuard VPN profile. Watch for welcome pack.	Required
14 to 16 Oct	In-person event at Strathmore University, Nairobi. Bring laptop + ID.	Event
Registration close	Not announced. Registration is open at jengasec.com/accounts/register/.	TBC

In one line: build and secure one JengaBank backend, ship it as a private linux/amd64 Docker image that reaches **Healthy** before **Tue 13 Oct, 2:00 PM EAT**, then defend it live, without patching, from 14 to 16 October.

PART 1

What the briefing covered

JengaSec is a full-spectrum enterprise cybersecurity simulation run by the Strathmore University Cybersecurity Club with Hack The Box, ISACA Kenya Chapter, Digital Africa and iLab Africa. Entry is free; level is medium to advanced.

The concept

- One fictional bank, **JengaBank**, run as two identical copies: **Enterprise A (ENT-A)** and **Enterprise B (ENT-B)**.
- Each enterprise has nine applications. Each app is built by one team in ENT-A and one in ENT-B, independently, and the two builds are compared.
- Three days, one attack-and-defence round per day: keynotes (Maarifa), the round (Morani Hunting Grounds), then executive review (Kikao).
- Everyone, Blue and Red, reaches the range only through a personal WireGuard VPN profile issued by JengaSec.

The four tracks

Track	Role
Application Blue (us)	Build and secure one JengaBank application, then defend it at the event.
Application Red	Attack the Blue-built apps, bug-bounty style. Qualify via the Hack The Box qualifier (11 to 12 Oct).
AI Defence Blue	Build an AI agent that reads the bank's telemetry (from our logs) and responds to attacks. Runs on their own machines.
AI Red	Attack the AI defence agents: make them miss attacks or block real customers.

No Cloud track this year. One person, one team, one side: nobody can be on both Blue and Red.

The nine applications

App	Name	App	Name
APP01	Internet Banking Portal	APP06	Internal Back-Office Portal
APP02	Mobile Banking API (an API, not a phone app)	APP07	Customer Support & Ticketing
APP03	Customer Authentication & Identity	APP08	Fraud & Transaction Monitoring (optional)
APP04	Payment Processing	APP09	Notifications & Statements (optional)
APP05	Loan Origination		

Your team page shows which enterprise and app you own. Full briefs are in the welcome pack.

Who can take part

- Teams of four, with at least one cybersecurity student or security specialist (marked on the team page).
- Aged 18 to 25 on the registration date, checked against ID. Students use their institutional email.
- Teams may mix universities. Mentors may advise but not write anything, and must be declared on the proposal cover page.
- Bring your own laptop and ID. No machines are provided.

PART 1 (CONTINUED)

What we must build and how it is checked

- The backend service in our brief, in any language or framework.
- Delivered as a **private Docker image for linux/amd64**. We do not upload source code.
- **Secure by design:** the proposal's threat model and security objectives should show up in the running app.
- **Observable:** every request writes a security log line.
- **Defensible:** once frozen we cannot patch; at the event we defend by detecting and responding.

The eight image checks

#	Check	Detail
1	linux/amd64	Apple Silicon builds ARM by default; use --platform linux/amd64.
2	Non-root user	No privileged mode, host networking or host folders.
3	One HTTP port	Declared in the submission form, e.g. 8080.
4	Health check	/healthz returns HTTP 200 when ready. Website also asks for /readyz and /openapi.json.
5	JSON logs to stdout	One JSON object per line, per request.
6	Fits the limits	2 CPU and 3 GB memory total, database included. Image under 2 GB.
7	Creates its own tables	Runs migrations and loads seed data on startup.
8	No internet at runtime	Outbound internet is blocked. Bundle everything in the image.

Logging contract

If it is not logged, our defence cannot be credited. One line per request on stdout, security events logged within 5 seconds. Required fields:

Field	Meaning
ts	Time in UTC, ISO 8601
src_ip	Client address as the app sees it
method, path, status	The HTTP request and our response code
user	Logged-in user, or empty
action	Short event name, e.g. login.failed, transfer.created, transfer.denied

Submitting: five steps

Step	What to do
1. Build	docker buildx build --platform linux/amd64, a new tag each time (v1, v2...), never latest.
2. Push privately	GitHub Container Registry (recommended) or Docker Hub. Visibility must be Private.
3. Read-only token	GitHub: classic token with read:packages only. Docker Hub: read-only token. Never a password.

Step	What to do
4. Fill the form	Team page → Submit application: image, username, token, port, health path, database yes/no.
5. Read the result	Pulled → Scanned → Deployed → Healthy. Healthy is a pass. Resubmit as often as needed before the freeze.

JengaSec pulls the image into its own registry, scans it, runs it on the range with PostgreSQL 16 if requested, and shows the scan report and app logs.

PART 1 (CONTINUED)

Troubleshooting, freeze and fair play

Common failures

You see	Likely cause	Fix
unauthorized / denied at Pulled	Wrong username, expired token, missing read:packages	New read-only token, update the form
exec format error	Image built for ARM (Apple Silicon)	Rebuild with --platform linux/amd64
Stuck at Deployed	Wrong port or health path, or crash on start	Check logs on the team page, then port and path
OOMKilled	Over the 3 GB memory limit	Reduce memory use
permission denied writing files	Non-root user cannot write there	Write to /tmp or a folder you chown
Critical vulnerabilities	Old base image or packages	Newer base image (e.g. -slim), rebuild
Rejected: image is public	Registry visibility is public	Set to Private, resubmit

At the freeze and on event days

- The last passing (Healthy) image is locked by its exact digest. Pushing afterwards changes nothing. **No Healthy image at freeze means the app cannot take part.**
- The database is snapshotted after the first healthy start at freeze and restored before each competition day.
- Red teams attack during a bounded window each day. No redeploying or live patching: watch logs, spot attacks, respond and report.
- How Blue teams are scored at the event is **to be confirmed** in the welcome pack.

Fair play

- Keep the image, token and brief private. Public images are rejected.
- No malware, miners or container-escape attempts. Every image is scanned; breaking this disqualifies the team.
- Made-up data only. No real personal data or hard-coded passwords for real services.
- The WireGuard profile is personal; never share it.
- AI tools are allowed for editing and diagrams if declared, but **not** for the threat model, architecture or security design.

Not announced yet

Full app briefs and published logging contract; Blue scoring; daily programme; prizes and speakers; meals, transport and accommodation; when registration closes; AI Action API details. Questions go to the support channel on jengasec.com with team code, image reference and a screenshot.

PART 2

How to maximise our chances of winning

The biggest risk is not a weak app; it is missing a Healthy image at the freeze. After that, credit goes to apps that are hard to break and easy to observe. Priority if time runs short: Healthy image > access checks and money logic > logging quality > self-defence features > polish.

1. Today (Sun 11 Oct): admin and de-risking

- Every member completes account, profile, ID and Code of Conduct. Captain marks the security specialist and deputy.
- Confirm our enterprise and app. Read the brief and the Image Submission Guide end to end.
- **Push a "hello world" image through the full pipeline tonight**: multi-stage build on a -slim or distroless base, --platform linux/amd64, non-root USER, /healthz + /readyz + /openapi.json, JSON logs to stdout, migrations and seed on startup.
- Ask organisers: how Blue is scored; whether extra log fields are allowed; whether PostgreSQL counts toward 3 GB; what src_ip looks like behind the range; how we read our logs live at the event.

2. Mon 12 Oct: build it secure

Every vulnerability is permanent after the freeze. What Red teams will try first on a banking backend:

Area	What to do
Object access (IDOR / BOLA)	The number one bank-app bug. Check ownership on every request: /accounts/{id}, /transfers/{id}, statements, tickets. Never trust client-supplied IDs.
Authentication	argon2 or bcrypt hashing; rate limiting and lockout; strong JWT secret generated at startup, pinned algorithm (reject none), short expiry, logout invalidation; generic error messages.
Authorisation	Server-side role checks on every admin or staff route. Attackers will call every endpoint listed in /openapi.json.
Money logic	Reject negative, zero, non-numeric or huge amounts. DB transactions with row locks against double-spend races. Idempotency keys. No self-transfer or transfer to unknown accounts. Enforce limits server-side.
Injection	Parameterised queries or ORM only. Schema-validate every input. Block mass assignment (ignore "role" or "balance" in request bodies).
Errors and leakage	No stack traces, debug endpoints or version headers. Security headers and a tight CORS policy.
Seed data	No weak or default passwords on seeded users, especially admins.
Uploads (if any)	Validate type and size, store safely, never use the client filename in paths.

Make the app defend itself, because there is no patching later: throttle or lock after repeated failed logins, flag transfer velocity and amount anomalies, reject suspicious payloads, log everything. Keep auto-blocking conservative; blocking legitimate customers likely counts against us.

3. Logging: our biggest lever

- One line per request, plus distinct security action names: login.failed, login.locked, authz.denied, transfer.denied, input.rejected, ratelimit.hit, token.invalid.
- Flush within 5 seconds: disable stdout buffering (e.g. PYTHONUNBUFFERED=1).
- Never log passwords, tokens or full secrets. Always fill src_ip and user accurately.
- Rich, accurate logs help us, the judges and the AI Defence teams who read our telemetry.

4. Tue 13 Oct morning: test, harden, freeze early

- Red-team ourselves with Burp or ZAP, sqlmap and ffuf. Swap IDs between two test users; fire parallel transfers at one balance.
- Scan locally with Trivy or Grype until there are zero criticals.
- Load-test inside 2 CPU / 3 GB. An OOMKill during the event is fatal.
- **Final Healthy submission by about 11 AM**, not 1:59 PM. New tag every time.
- Documents: the threat model (STRIDE works) and security objectives must match the frozen app. Written by us, not AI.

5. Event days (14 to 16 Oct): live defence

- Prepared tooling: jq filters or a small script grouping logs by action, src_ip and user, flagging spikes.
- Incident report template: time, source, technique (MITRE ATT&CK ID), endpoint, evidence log lines, response.
- Report fast and with evidence; detection and reporting is how a frozen app earns credit.
- Logistics: charged laptops, ID, WireGuard profile downloaded and tested beforehand, power bank.

PART 3

How our four members split the work

Split by ownership area so nobody collides in the codebase, and sync at fixed checkpoints.

Roles

Member	Role	Owns
A: _____	Captain + Ops / Platform	Admin checklist, Dockerfile, GHCR and token, submission form, migrations and seed, resource limits, image scanning, **the freeze decision**.
B: _____	Builder: Identity & Access	Login, sessions / JWT, hashing, lockout and rate limits, roles, and the shared **authorisation helper** (ownership and permission checks).
C: _____	Builder: Core Business Logic	The app's main features (transfers, loans, tickets per the brief), input validation, money and transaction safety, DB schema. Uses B's helper rather than writing its own checks.
D: _____	Security Lead (registered security specialist)	Threat model and proposal documents, logging contract and middleware, security review of B and C's code, self red-teaming, event-day playbook. Does not build features.

Timeline by person

When	A	B	C	D
Sun 11 Oct (tonight)	Team admin complete. Hello-world image to Healthy.	Read brief. Design users, roles, sessions.	Read brief. Draft DB schema and endpoint list.	Read brief. Start threat model. Write logging middleware and action names.
Mon 12 Oct	Build script, image under 2 GB, Trivy scan, keep main Healthy. Profiles done by 23:59.	Auth and authorisation fully working, plus lockout and rate limits.	Core features on B's helper; transactions and row locks on money paths.	Review B and C's code as it lands. Write threat model and security objectives.
Tue 13 Oct, to ~9 AM	Load test (2 CPU / 3 GB).	Fix gaps from D's review.	Fix gaps from D's review.	Attack the app: IDOR, races, SQLi, JWT, mass assignment. File findings.
Tue 9 to 11 AM	Final scan, zero criticals.	Fix only D's findings. No new features.	Fix only D's findings. No new features.	Finalise and upload documents.
Tue ~11 AM	**Submit final image, confirm Healthy.**	Stop coding.	Stop coding.	Check documents match the frozen app.
Tue 11 AM to 2 PM	Buffer for resubmits.	Prepare event tooling.	Prepare event tooling.	Incident report template.

Sunday-night sync (all four, required): agree the API spec (endpoints, request and response shapes), DB schema, repo layout and log format. After this, work runs in parallel. **Monday 6 PM checkpoint:** merge and submit for a Healthy image with real features.

Event days: role switch

Member	Event role
D	Incident lead: decides what is an attack and maps it to MITRE ATT&CK.
B	Log watcher: authentication events (failed logins, token abuse, privilege attempts).
C	Log watcher: business-logic events (transfer anomalies, validation rejections).
A	Reporter and liaison: writes and submits reports, handles organisers, Live Status and scorecard.

Each watcher monitors the code they wrote, because they know what normal looks like.

Ground rules

- No security-critical code merges without D's review.
- A owns the freeze. When A says stop, everyone stops.
- A short check-in every 4 to 6 hours: done, blocked, next.
- One shared doc for decisions (endpoints, action names, secrets handling) so nobody guesses.

Resources

Where	What
jengasec.com	Overview, dates and FAQ
jengasec.com/tracks/application-blue/	Our track: brief, sections, rules
jengasec.com/prepare/	Rules and checklist
Team page (login)	Our app, Submit application, results, exact freeze deadline
Image Submission Guide	Step-by-step build, push, token and form
Support channel on jengasec.com	Questions about our submission